🚀 CONSTRUCCIÓN DE PRODUCTO & CLOUD AUTOMATION

Construcción del MVP & Automatizaciones Cloud

De la idea a un SaaS funcional con IA en producción, persistencia gratuita con Keep-Alive en cron-job.org y despliegue global en Vercel.

⚡ Prototipado Asistido

Antigravity + Prompt Maestro para UI responsiva en 2 columnas, formulario reactivo y orquestación Gemini con JSON estricto.

🗄️ Supabase Postgres & RLS

Persistencia relacional segura con Row Level Security y políticas públicas para captura instantánea de cotizaciones y leads.

🌐 Vercel & Keep-Alive

Despliegue continuo con Git SSH y el hack de persistencia 24/7 con cron-job.org para evitar la pausa del tier gratuito.

🎙️ GUION DEL INSTRUCTOR

¡Bienvenidos a la Clase 2 de Vibecode Lab! Hoy pasamos de la teoría y la configuración a la acción pura. En esta sesión no solo crearemos la interfaz de nuestro SaaS; construiremos un flujo completo de punta a punta: formulario reactivo, orquestación de IA con Gemini en JSON estricto, guardado en base de datos Postgres con Supabase, despliegue en Vercel y un truco de ingeniería para que su aplicación nunca se duerma sin pagar un solo dólar.

La Filosofía del Vibecoding en Producción

No programes línea por línea: diseña especificaciones claras y orquesta agentes de IA.

1. **El Enfoque Tradicional vs. Vibecoding**
   Programar a mano un cotizador con validaciones, cálculos y bases de datos tomaba 40 horas. Con Antigravity y especificaciones rigurosas lo construyes y testeas en menos de 90 minutos.

2. **El Caso de Estudio: VibeCargo**
   Un cotizador inteligente de fletes y mudanzas que calcula costos en pesos chilenos (CLP) según inventario y pisos por escalera, convirtiendo leads directamente a WhatsApp.

3. **Patrón Arquitectónico Universal**
   Este mismo patrón sirve para cotizadores inmobiliarios, calculadoras de consultoría, presupuestos médicos o diagnósticos legales: Formulario → LLM → DB → Conversión.

💡 Regla de Oro del Vibecoder Profesional

"La inteligencia artificial no adivina lo que tienes en la cabeza. La robustez y elegancia de tu software es un reflejo exacto de la precisión de tus especificaciones técnicas."

> 🎯 **Tu Rol Hoy:** Actuar como Arquitecto de Software y Product Manager. Tú tomas las decisiones estructurales; Antigravity genera el código y tú lo auditas en tiempo real en tu navegador.

🎙️ GUION DEL INSTRUCTOR

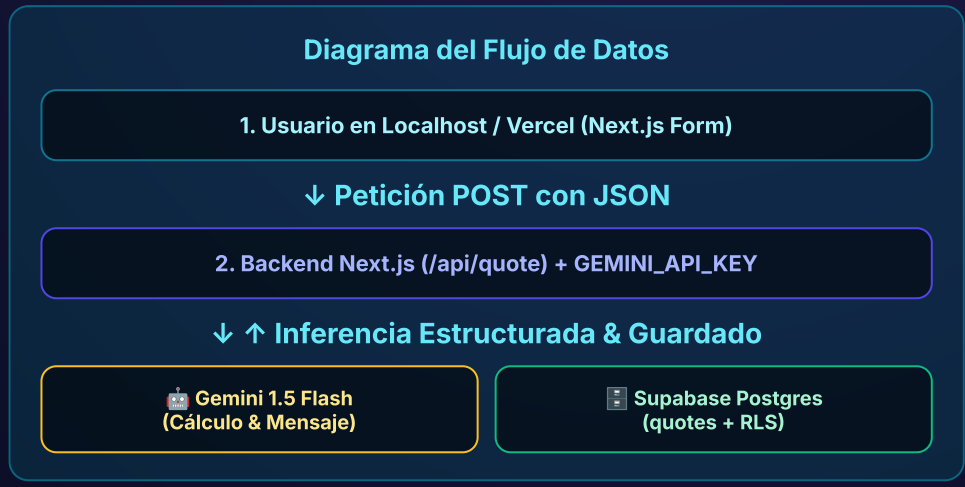
Quiero que cambiemos el chip. Si intentas escribir cada componente de React desde cero, te vas a desgastar en la sintaxis. Nuestro superpoder es saber pedir exactamente lo que el producto necesita: estados visuales, control de errores, experiencia de usuario y arquitectura. Hoy usaremos VibeCargo como ejemplo, pero recuerden que cada uno puede personalizar su tabla y su cotizador según su nicho.

Arquitectura Técnica: El Flujo de Datos del MVP

Una arquitectura serverless desacoplada donde la seguridad y la velocidad son prioridad.

El cliente interactúa únicamente con el Frontend. Todas las operaciones pesadas y confidenciales (cálculos de costos y claves privadas de IA) ocurren de forma segura en el servidor.

1. **Captura Reactiva (Frontend Next.js)**
   Formulario moderno en 2 columnas: captura origen, destino, pisos por escalera e inventario con validación en vivo.
2. **Orquestación Serverless (/api/quote)**
   Ruta backend segura que procesa la petición y llama a la API de Google Gemini sin exponer las credenciales en el navegador.
3. **Persistencia & Cierre en WhatsApp**
   El backend guarda el JSON en Supabase (Postgres) y retorna el desglose para abrir la API de WhatsApp con el mensaje prellenado.



🎙️ GUION DEL INSTRUCTOR

Miren con atención esta arquitectura. ¿Por qué no llamamos a Gemini directamente desde el botón en React? Porque si pones tu API Key en el frontend, cualquier persona puede abrir las herramientas de desarrollador, robarte la clave y gastarte el saldo. Siempre, grábenselo con fuego: las llamadas a IA y las escrituras críticas se hacen a través de una API Route protegida en Next.js.

🤖 Prompt Maestro para Antigravity (app/page.js)

Copia y pega este prompt en el chat de Antigravity para construir la interfaz integral en minutos.

PROMPT INTEGRAL DE CONSTRUCCIÓN DE INTERFAZ — Copiar y Pegar en Antigravity

```
"Actúa como un arquitecto frontend y fullstack senior de Next.js (App Router). Construye en app/page.js la interfaz completa, moderna y 100% responsiva para el cotizador inteligente 'VibeCargo': 1. DISEÑO & ESTILO (Tailwind CSS): - Hero header con gradiente moderno, badge animado ('Cotizador IA en Vivo') y diseño limpio. - Layout en 2 columnas en desktop (Formulario a la izquierda y Tarjeta de Resultados a la derecha). 2. FORMULARIO REACTIVO (Campos de Entrada): - Dirección de Origen y Destino (inputs con iconos). - Pisos por escalera (contador interactivo con botones +/- y visualizador numérico). - Descripción de Muebles / Ítems (textarea con placeholders sugeridos). - Botón de Envío con estados visuales: Idle ('Calcular Cotización 🚀'), Loading ('Calculando con Gemini... ⏳') y Disabled. 3. CONEXIÓN BACKEND (Llamada Asíncrona): - Realiza fetch POST a '/api/quote' enviando los datos del formulario en JSON con manejo limpio de errores. 4. TARJETA DE RESULTADOS & ACCIÓN WHATSAPP: - Renderiza tabla de costos ('data.breakdown') formateada en pesos chilenos ($XX.XXX CLP) y precio final destacado. - Botón verde destacado 'Compartir por WhatsApp' que abra 'https://wa.me/?text=...' con el texto codificado."
```

✓ 2 Columnas Desktop
Formulario a la izquierda y desglose visual a la derecha.

✓ Estados Reactivos
Loading spinner, botones deshabilitados y feedback de error.

✓ Moneda & WhatsApp
Formato numérico CLP y deep link encodeURIComponent para chat directo.

🎙️ GUION DEL INSTRUCTOR

Miren la estructura de este prompt. Tiene 4 bloques perfectamente ordenados: Diseño, Formulario, Conexión Backend y Resultados con WhatsApp. Cuando le das una orden así a Antigravity, no genera código al azar; crea exactamente los estados de React, los componentes de Tailwind y los manejadores de eventos necesarios. Copien este prompt y pónganlo a prueba en sus pantallas ahora mismo.

Orquestación de IA: Gemini con JSON Estricto

Por qué el modo JSON estructurado es obligatorio para construir software en producción.

❌ **El Problema del Texto Libre**
Si le pides a un LLM que responda en texto común, te agregará saludos, formato markdown (```json) o aclaraciones que romperán el parseo de tu frontend en tiempo de ejecución.

✅ **La Solución: System Prompt con Esquema Rígido**
Forzamos a Gemini a responder única y exclusivamente un objeto JSON válido con los campos exactos: `breakdown` (array de ítems), `estimated_total` (número) y `whatsapp_message` (string listo).

🔒 **Seguridad del Lado del Servidor**
La API Key vive en `.env.local` y Vercel. El frontend solo recibe el JSON procesado y formateado, protegiendo tu cuenta de Google Cloud.

```ts
// app/api/quote/route.ts
import { GoogleGenerativeAI } from "@google/generative-ai";
import { NextResponse } from "next/server";

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY!);

export async function POST(req: Request) {
  const body = await req.json();
  const model = genAI.getGenerativeModel({
    model: "gemini-1.5-flash",
    generationConfig: { responseMimeType: "application/json" }
  });

  const prompt = `Calcula cotización en CLP para:
  Origen: ${body.origin}, Destino: ${body.destination},
  Pisos: ${body.floors}, Ítems: ${body.items}.
  Responde SOLO este esquema JSON:
  {
    "breakdown": [{"item": string, "cost": number}],
    "estimated_total": number,
    "whatsapp_message": string
  }`;

  const result = await model.generateContent(prompt);
  return NextResponse.json(JSON.parse(result.response.text()));
}
```

🎙️ GUION DEL INSTRUCTOR

Fíjense en la propiedad `responseMimeType: "application/json"`. Esto es fundamental. Al activar esto en Gemini 1.5, el modelo entra en modo de gramática restringida y garantiza matemáticamente que la respuesta sea un JSON parseable. Se acabaron los errores de `SyntaxError: Unexpected token in JSON at position 0`. Así es como programan los ingenieros senior de IA.



Break & Nivelación de Entornos (10 Minutos)

Aseguremos que todos tengamos el cotizador funcionando localmente antes de conectar la base de datos y la nube.

📋 Checklist de Verificación Local

- ✓ **1. Servidor Dev Activo**
  Comando `npm run dev` ejecutándose sin errores en tu consola.
- ✓ **2. Localhost:3000 Abierto**
  Ver la interfaz moderna de VibeCargo con su formulario en el navegador.
- ✓ **3. Cotización Calculada**
  Ingresar datos de prueba y comprobar que Gemini devuelve el desglose en CLP.
- ✓ **4. Botón de WhatsApp**
  Clic en el botón verde y confirmar que abre WhatsApp con el mensaje prellenado.

🆘 ¿Te quedaste atascado?

No te preocupes. Los problemas más comunes en esta etapa son:

- Falta la variable `GEMINI_API_KEY` en tu archivo `.env.local`.
- No instalaste dependencias: corre `npm install`.
- Puerto 3000 ocupado: prueba en `http://localhost:3001`.

Levanta la mano en Zoom o avísanos por el chat para ayudarte en vivo.

🎙️ GUION DEL INSTRUCTOR

Tomamos 10 minutos de break. Aprovechen para servirse agua, despejarse y verificar su checklist. Si alguien tiene un error en la consola o no le calcula la cotización, avísenos ahora mismo. En la segunda mitad de la clase subiremos este proyecto a su propio GitHub, lo desplegaremos en Vercel y configuraremos la persistencia automática de Supabase.

Persistencia en Supabase: Tabla quotes & RLS

En aplicaciones web la anon key es pública: la seguridad vive en las políticas de Postgres.

1. **Postgres Gestionado en la Nube**
   Supabase te entrega una base de datos Postgres real con API REST y GraphQL instantánea lista para producción.

2. **¿Por qué Row Level Security (RLS)?**
   Por defecto, Postgres bloquea todas las peticiones externas. RLS te permite definir exactamente qué filas puede leer o insertar un usuario anónimo o autenticado.

3. **Adaptabilidad por Alumno**
   Si tu proyecto no es de fletes, cambia `quotes` por el nombre de tu modelo: `leads`, `tickets`, `reservas` o `pedidos`.

```sql
-- Supabase SQL Editor: Copiar y clic en "RUN"
CREATE TABLE IF NOT EXISTS public.quotes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  origin_address TEXT NOT NULL,
  destination_address TEXT NOT NULL,
  floors_stairs INT DEFAULT 0,
  items_description TEXT NOT NULL,
  estimated_total NUMERIC(10,2) NOT NULL,
  breakdown_json JSONB NOT NULL,
  whatsapp_message TEXT NOT NULL,
  status TEXT DEFAULT 'pending'
);

-- Habilitar Seguridad RLS y Políticas de Acceso:
ALTER TABLE public.quotes ENABLE ROW LEVEL SECURITY;

CREATE POLICY "permitir_insercion_publica"
ON public.quotes FOR INSERT TO anon, authenticated
WITH CHECK (true);

CREATE POLICY "permitir_lectura_publica"
ON public.quotes FOR SELECT TO anon, authenticated
USING (true);
```

🎙️ GUION DEL INSTRUCTOR

Miren las dos políticas que creamos abajo: `permitir_insercion_publica` y `permitir_lectura_publica`. Si no ejecutan esas dos líneas, cuando el formulario intente guardar una cotización, Supabase responderá con un error 42501 de violación de RLS. Con este script aseguramos que cualquier visitante del formulario pueda registrar su cotización de forma transparente.

De Local a la Nube: GitHub (SSH) & Despliegue Vercel

El estándar de la industria: cada cambio empujado a GitHub se despliega automáticamente en la web.

1 Crear Repo Vacío en GitHub

Entra a `github.com/new`, dale un nombre a tu proyecto (ej: `mi-mvp-cotizador`) y déjalo **público sin README**.

2 Re-vincular por SSH en Terminal

Cambia el origen remoto a tu cuenta y sube tu código:

```
git remote set-url origin git@github.com:TU_USUARIO/TU_REPO.git
git add .
git commit -m "feat: MVP cotizador inteligente con IA"
git push -u origin main
```

3 Importar Proyecto en Vercel

Inicia sesión en Vercel con GitHub ➔ Clic en **Add New... ➔ Project** ➔ Selecciona tu repo recién subido.

🔑 Variables de Entorno en Vercel

Antes de hacer clic en **Deploy**, debes agregar estas 3 variables en la sección *Environment Variables* de Vercel:

`GEMINI_API_KEY`
Clave privada obtenida de Google AI Studio (aistudio.google.com).

`NEXT_PUBLIC_SUPABASE_URL`
Project Settings ➔ API ➔ Project URL en Supabase.

`NEXT_PUBLIC_SUPABASE_ANON_KEY`
Project Settings ➔ API ➔ anon public key en Supabase.

🎙️ GUION DEL INSTRUCTOR

Este es el momento mágico en que su software sale de su computadora y pasa a estar disponible para todo el planeta. Usar SSH garantiza que no tengan que estar escribiendo contraseñas cada vez que suban cambios. Y al conectar Vercel con GitHub, cada mejora que le pidan a Antigravity y suban con `git push` se convertirá en un despliegue automático en segundos.

El Hack de Persistencia Gratuita: Keep-Alive + cron-job.org

Cómo evitar que Supabase pause tu proyecto gratis después de 7 días de inactividad.

⚠️ El Problema de la Pausa en Free Tier

Supabase Free duerme automáticamente cualquier base de datos que pase 7 días consecutivos sin consultas. Si un cliente entra el día 8, la app fallará.

```ts
// app/api/keep-alive/route.ts
import { NextResponse } from "next/server";
import { getSupabaseClient } from "@/lib/supabase";

export const dynamic = "force-dynamic";
export const revalidate = 0;

export async function GET() {
  const supabase = getSupabaseClient();
  // Consulta ultra ligera para reactivar el motor Postgres
  const { error } = await supabase.from( "quotes" ).select("id").limit(1);
  // Salvavidas: Error 42P01 confirma que Postgres respondió activo
  const isAlive = !error || error.code === "42P01";
  return NextResponse.json({ ok: isAlive, status: "alive" });
}
```

🌐 Automatización Externa con cron-job.org

1. **Crear Cuenta Gratuita**
   Ingresa a `https://cron-job.org` y regístrate en 30 segundos.
2. **Crear Nuevo Cronjob**
   URL: `https://tu-proyecto.vercel.app/api/keep-alive`
3. **Programar Frecuencia**
   Seleccionar: **Every 3 days (04:00 UTC)** con notificación por email ante fallos.

💰 Costo Total: $0 USD / mes

Tu base de datos Postgres permanecerá 100% activa los 365 días del año sin costos mensuales.

🎙️ GUION DEL INSTRUCTOR

Este es uno de los hacks más celebrados por nuestros alumnos. Para qué pagar $25 dólares al mes por una base de datos cuando estás validando una idea en fase temprana, si puedes programar un endpoint ligero de 10 líneas y un cron gratuito que le haga un 'ping' de vida cada 3 días. Ingeniería inteligente es lograr el máximo uptime con el mínimo gasto.

🔧 Troubleshooting: Errores Frecuentes en Vivo

Los 4 problemas clásicos de la Clase 2 y cómo resolverlos en menos de 1 minuto.

1. SSH Permission Denied (publickey)

Causa: Git no encuentra tu llave privada o no agregaste la clave pública a tu cuenta de GitHub.

```
eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519
ssh -T git@github.com # Debe decir: "Hi username! You've successfully authenticated"
```

2. Error 401 / Invalid API Key de Gemini

Causa: Clave copiada con espacios accidentales o variable no guardada en el entorno activo.

Solución: Revisa `.env.local`. En Vercel: ve a *Settings* ➔ *Environment Variables*, actualiza la clave y haz un **Redeploy**.

3. Error 42501 (Violates RLS Policy)

Causa: La tabla existe en Supabase pero no creaste la política pública de inserción (INSERT).

Solución: Abre el SQL Editor en Supabase y ejecuta: `CREATE POLICY "permitir_insercion" ON quotes FOR INSERT WITH CHECK (true);`

4. Build Failed en Vercel

Causa: Diferencias de mayúsculas/minúsculas en rutas de archivos entre Linux y Mac/Windows.

Solución: Revisa los imports (ej: `@/components/QuoteCard` vs `quotecard`) y asegúrate de que coincidan exactamente en el disco.

🎙️ GUION DEL INSTRUCTOR

Miren esta diapositiva y guárdenla. El 99% de los alumnos que se traban hoy caen en uno de estos 4 puntos. No entren en pánico. Desarrollar software profesional consiste precisamente en saber diagnosticar: si es error de llave, de credencial de IA, de permiso SQL o de compilación de Vercel. Cada error tiene una solución exacta en un solo comando.

🏆

¡Misión Cumplida! Tu Primer SaaS en Producción

En solo 2 clases pasaste de cero a desplegar un MVP con IA generativa, base de datos en la nube y persistencia 24/7.

✅ Entregables de la Clase 2

- Repositorio propio público en GitHub (SSH).
- SaaS desplegado en `tu-proyecto.vercel.app`.
- Formulario cotizador conectado a Gemini.
- Tabla Supabase con RLS capturando cotizaciones.

📋 Tarea para esta Semana

- Configurar Cronjob Keep-Alive en cron-job.org.
- Personalizar textos e imágenes para tu nicho.
- Compartir 3 cotizaciones reales por WhatsApp.
- Publicar tu link en el Discord de Vibecode Lab.

🔮 Próxima Sesión: Clase 3

- Autenticación segura con Google OAuth.
- Sincronización auth.users ➔ public.profiles.
- Políticas RLS multi-tenant para usuarios privados.
- Protección contra enumeración y validación Zod.

🎙️ GUION DEL INSTRUCTOR

¡Felicitaciones a todos! Siéntanse muy orgullosos del progreso de hoy. Muchos pasan meses aprendiendo teoría sin tener nada público. Ustedes, en la segunda semana, ya tienen una aplicación web moderna desplegada en producción, consumiendo modelos de lenguaje, guardando datos en Postgres y automatizada para no caerse nunca. ¡Nos vemos en la Clase 3 para blindar nuestro SaaS con Google OAuth y Autenticación Segura!